

day 17

day 3 node

📌 Obsidian Notes: Express.js MVC Architecture & Local File Storage

💡 لماذا نستخدم هذا الـ **Structure**؟ (Separation of Concerns)

بدلاً من كتابة التطبيق كاملاً داخل ملف `index.js` واحد، نقوم بتقسيم الكود إلى طبقات (Layers):

1. **Routing Layer**: وتوجيهها HTTP Requests مسؤولة فقط عن استقبال الـ.
2. **Controller Layer**: Responses. والتعامل مع البيانات وإرجاع الـ Business Logic مسؤولة عن الـ.
3. **Data Layer**: (`users.json` و `posts.json`) الملفات التي تمثل الداتابيز.
4. **Middleware Layer**: فحص الأخطاء والمسارات غير الموجودة.
5. **Utils Layer**: (DRY Principle) أدوات مساعدة معزولة لحماية الكود من التكرار.

1. الملف الرئيسي (`index.js`)

هو الـ Entry Point السيرفر، ومسؤول فقط عن تجميع التطبيق وتجهيز الـ Middlewares العامة:

- `app.use(express.json())` ويضعها في JSON يفكك البيانات المبعوثة بصيغة : `req.body`.
- `app.use(userRouter)` يربط مسارات المستخدمين بالسيرفر الرئيسي :
- `app.use(notFoundHandler)` يُبحث لمسار غير معرف مسبقاً Request يلتقط أي :
- `app.use(errorHandler)` Controllers. الميدل وير الأخير لالتقاط أي خطأ يتربى داخل الـ :

2. طبقة الأدوات المساعدة (`/utils`)

أ) `utils/file-operation.js` (التعامل مع الملفات)

- الهدف: تجميع عمليات الـ I/O في مكان واحد بدلاً من تكرار `fs.readFile` و `JSON.parse` داخل كل Controller.
- `readFile` : `JSON.parse` بـ Object/Array وتحواله لـ JSON تقرأ ملف الـ :
- `writeFile` : `JSON.stringify` بـ JSON string تحول البيانات لـ :

ب) `utils/AppError.js` (كلاس الأخطاء المخصص)

- الهدف: إنشاء Operational Errors مخصصة نحدد فيها الـ Message والـ StatusCode ليمرر تلقائياً لـ Error Middleware.
- `Error.captureStackTrace` : وتحدد السطر الفعلي الذي تسبب بالخطأ Stack Trace تمنع تلوث الـ :

● 3. طبقة الميدل وير (/middleware)

أ) middleware/not-found.js

دالة بسيطة تُستدعى إذا لم يطابق الـ Request أي Endpoint معروف ويرجع 404 Not Found .

ب) middleware/error.js

ميدل وير مركزي (Central Error Handler). يفحص هل الخطأ ناتج من `AppError` ليرجع الـ Status Code المحدد والرسالة، أم خطأ غير متوقع سيرفر ليُرجع `Internal server error 500`.

● 4. طبقة الـ Controllers والروابط المتبادلة (/controllers)

أ) controllers/user.js (إدارة المستخدمين)

- `createUser`: يقرأ الملف ليفحص تكرار الإيميل (name , age , email) يفحص الحقول المطلوبة : `users.some` (201) `Created` . لمنع الإيميلات المكررة، ثم يضيف المستخدم ويرجع 201 (`users.some`)
- `getAllUsers & getOneUser`: جلب البيانات وتصفياتها بالـ ID.
- `updateUserPutMethod vs upadateUserPatchMethod`:
 - `PUT`: `id` كاملاً بما فيه `Object` يشترط ارسال الـ `id` .
 - `PATCH`: لدمج البيانات `...body, ...user` يتطلب فقط الحقول المراد تغييرها ويستخدم `PATCH`.
- `deleteUser (Cascading Delete)`: يسمح المستخدم من `users.json` ويقرأ `posts.json` `deleteUser` . لتلقائياً `userId` البوستات التابعة لهذا الـ `userId` .

ب) controllers/post.js (إدارة البوستات والربط)

- `createPost (Validation Check)`: ليتأكد أولاً أن الـ `users.json` قبل إضافة البوست، يقرأ ملف `users.json` . `createPost` (`Validation Check`)

📌 ملاحظات ونكات هامة (Important Notes & Edge Cases)

⚠️ ⚠️ ملحوظة مهمة على الأكواد المرفقة (Fixes & Improvements):

1. الخطأ الإملائي في الاستدعاء (Import Spelling Mismatch): في ملف `controllers/user.js` مكتوبة المسار: `utils/file-opration/..` . بينما اسم الملف في الـ `Utils` مكتوب `file-operation.js` . تأكدي من توحيد الاسم بـ `file-operation` لتجنب خطأ `Cannot find module` .
2. استخدام `Async/Await` مع `WriteFile`: في `controllers/user.js` عند استدعاء `writeFile(userFile, users)` يجب إضافة `await` قبلها للتأكد من اكتمال حفظ الملف قبل إرسال الـ `Response` للعميل.

3. تحويل الأرقام (ID Comparison): تم استخدام `()toString` دائماً عند مقارنة الـ IDs مثل `req.params.id` يستقبل دائماً كـ `String` بينما في الملف قد يكون `Number`.

📌 Obsidian Notes: Express.js & REST API Architecture

المفهوم الأساسي

بدلاً من كتابة كل كود السيرفر في ملف واحد كبير، نستخدم نظام **MVC (Model-View-Controller)** لتقسيم الكود. الهدف إن كل ملف يكون مسؤول عن حاجة واحدة بس (Separation of Concerns).

1. هيكل المشروع (MVC Directory Structure)

- **index.js**: الـ Entry Point الذي يستقبل الـ Requests السيرفر، ببستقبل الـ `express.json()`.
- **routers/**: مسؤولة بس عن توجيه الـ (GET /users , POST /users) زي Endpoints المسارات والـ Request للـ Controller المناسب.
- **controllers/**: البيعد يقرأ ويعدل في الملفات أو الداتابيز، وبيجهز الـ (الحقيقي Logic قلب التطبيق! هو اللي فيه الـ Response).
- **model/**: (users.json و posts.json زي JSON حالياً ملفات) مكان تخزين البيانات : `middleware/` والـ `notFoundHandler` زي الـ `errorHandler` اللي في النص Handlers الفلاتر والـ `utils/` وديوال الـ `File Operation` وديوال الـ `Custom Error` `AppError` زي الـ الأدوات المساعدة المكررة :

2. طريقة التعامل مع البيانات والتعديل عليها

عشان نعمل CRUD كامل على ملفات الـ JSON، بنستخدم دالتين أساسيتين:

1. **readFile**: JS نقدر نعدل عليه في Object أو Array عشان نحوله لـ `JSON.parse` وبتعمله `JSON` بتقرأ ملف الـ `readFile`.
2. **writeFile**: عشان تحفظها جوه الملف تاني `JSON.stringify` بعد التعديل وبتعملها `Array` بتاخذ الـ `writeFile`.

الفرق بين الـ PUT والـ PATCH:

- **PUT**: (Full Replace) كامل بـ كل الحقول Object لازم العميل يبعث الـ (name , age , email).
- **PATCH**: (Partial Update) تحديث جزئي : (مثلاً email) العميل بيبعث الحقول اللي عايز يغيره بس. للدمج Spread Operator : { ...user, ...body }.

3. طريقة إدارة الأخطاء (Centralized Error Handling)

بدل ما نكتب `try/catch` ونقعد نبعث `res.status().json()` في كل مكان:

1. **AppError (Custom Error Class):**

كلاس بسيط بيخلينا نحدد الـ Message والـ StatusCode في مكان واحد:

JavaScript

```
throw new AppError("User not found", 404);
```

2. errorHandler Middleware:

ميدل وير بيتخط في آخر `index.js` خالص، وظيفته يلقف أي خطأ يتربى بـ `throw` أو يتبعث لـ `next(err)` ويرجع الـ `Response` بالشكل المضبوط:

JavaScript

```
if (err instanceof AppError) {  
  return res.status(err.statusCode).json({ message: err.message });  
}
```

4. العلاقات بين البيانات (One-to-Many Relationship)

- المفهوم: الـ User الواحد عنده أكثر من (One User -> Many Posts).
- التطبيق العملي:
 - جوه كل Post، بنحط حقل اسمه `userId` بيشير للـ `id` بتاع المستخدم اللي كتبه.
 - التحقق قبل الإضافة: لما نيجي نكريت Post جديد، بنقرأ ملف الـ Users الأول عشان نتأكد إن الـ `userId` ده موجود فعلاً!
 - الحذف المتتابع (Cascading Delete - Bonus): لما نيجي نمسح User، بنقرأ ملف الـ Posts ونمسح كل البوستات اللي الـ `userId` بتاعها بيساوي ID الـ User اللي اتمسح.

ملخص الأوامر والـ Endpoints المطلوبة للتاسك 📋

Method	Endpoint	Description
GET	/users	جلب كل المستخدمين
GET	/users/:id	جلب مستخدم محدد بـ ID
POST	/users	إضافة مستخدم جديد
PUT	/users/:id	تعديل كامل بيانات المستخدم
DELETE	/users/:id	حذف المستخدم (وحذف بوستاته تلقائياً)
GET	/posts	جلب كل البوستات
POST	/posts	إضافة بوست جديد (بشرط وجود الـ userId)

أسئلة إنتر views متوقعة (Interview Q&A) ?

② **Q1: Why do we place `express.json()` middleware before the routers in `index.js` ?** الإجابة: لأن `express.json()` يقوم بعمل `parsing` للـ `Request Body` المبعوث بصيغة JSON، سنكون قيمة `Routers` إذا وضعت بعد الـ `req.body` وتخزينه في `JavaScript Object` وتحويله لـ `JSON` داخل الـ `Controllers`. `req.body` `undefined` بـ

② **Q2: What is the main structural difference between PUT and PATCH HTTP methods?** الإجابة:

- **PUT:** يفترض إرسال المورد بكافة حقوله، وإذا غاب حقل قد يمحي أو (Full Replacement) استبدال كلي للمورد `null` يعوض بـ.
- **PATCH:** يستقبل فقط الحقول المراد تغييرها ويحافظ على بقية البيانات كما (Partial Update) تعديل جزئي للمورد هي.